Classification and Evaluation Metrics

Lesson 4 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

23 October 2026 · reading time about 80 minutes

Contents

Lesson plan . . . 2
1. From a number to a decision . . . 2
    1.1 The dataset, and why it is synthetic . . . 3
2. The model: from a line to a probability . . . 3
    2.1 What goes wrong with a straight line . . . 3
    2.2 The sigmoid . . . 4
    2.3 Odds and log-odds: why this particular curve . . . 5
    2.4 Reading the coefficients . . . 6
3. Cross-entropy, derived from maximum likelihood . . . 7
    3.1 The idea before the algebra . . . 7
    3.2 The likelihood . . . 7
    3.3 Why we take the logarithm . . . 8
    3.4 The gradient, and a small miracle . . . 8
4. Why not squared error . . . 9
    4.1 The cost of being confidently wrong . . . 9
    4.2 The sharper problem: the gradient dies . . . 10
    4.3 Convexity . . . 11
5. The trap: accuracy . . . 11
    5.1 The number to remember from this lesson . . . 11
    5.2 Why this keeps happening . . . 12
6. The confusion matrix and what is built on it . . . 13
    6.1 Four numbers . . . 13
    6.2 Precision and recall . . . 13
    6.3 F1, and why the harmonic mean . . . 14
    6.4 Averaging across classes . . . 15
7. The threshold is a decision . . . 15
    7.1 Nothing about the model changes along this axis . . . 15
    7.2 Choosing the threshold from costs . . . 17
8. ROC and AUC . . . 19
    8.1 The curve . . . 19
    8.2 What the area means . . . 21
    8.3 Where ROC misleads . . . 21
9. Class weights and resampling . . . 22

10. More than two classes . . . 23
10.1 Softmax . . . 23
10.2 Metrics with three classes . . . 24
11. Summary . . . 25
Homework . . . 25
Notation used in this lesson . . . 25

Lesson plan

Time	Segment	Material
0:00–0:12	Exercise 3 discussed; from numbers to decisions	Slides 2–6
0:12–0:35	Logistic regression: sigmoid, odds, coefficients	Slides 7–12
0:35–0:52	Why the cost had to change	Slides 13–18
0:52–1:15	Notebook 1, and the decision boundary	Slides 19–20
1:15–1:25	**Break**	Slide 21
1:25–1:45	The accuracy trap and the confusion matrix	Slides 22–25
1:45–2:05	Precision, recall, F1, and what to report	Slides 26–29
2:05–2:30	Notebook 2, and the threshold sweep	Slides 30–33
2:30–2:50	Receiver operating characteristic (ROC) curves, the area under them, and where they mislead	Slides 34–40
2:50–3:00	Cost-optimal thresholds; multiclass; homework	Slides 41–49
	Total	**180 minutes**

---

1. From a number to a decision

Lesson 3 predicted a quantity: given a house, how many euros. This lesson predicts a **category**: given a disk drive, will it fail in the next thirty days?

The change of question forces two changes of machinery, and it is worth being clear at the outset that they are separate.

The model has to change, because a straight line is unbounded and a probability is not. Section 2 fixes that with the sigmoid.

The evaluation has to change, because "how far off were we, on average" is meaningless when the answer is yes or no. Sections 5 to 8 are about that, and they occupy more of this lesson than the model does. That proportion is deliberate: fitting a classifier takes one line of scikit-learn, and knowing whether it is any good is where the actual difficulty lives.

1.1 The dataset, and why it is synthetic

Everything below uses 8,000 disk drives from a data centre. Each drive reports six SMART counters — **Self-Monitoring, Analysis and Reporting Technology**, the diagnostic statistics a drive keeps about itself — and a label saying whether it failed within thirty days.

306 of the 8,000 drives failed. That is 3.8%.

The data is generated by `Notebooks/disk_data.py`, and we know the coefficients that produced the labels. That is the same choice as lesson 3, for the same reason: a real dataset lets you admire an estimate, and a synthetic one lets you check it. One column, `seek_error_rate`, was generated with a coefficient of **exactly zero** — a plausible-looking counter with no connection to failure. It is there to be found out.

The counters are read on a logarithmic scale. Before the labels are generated, `disk_data.py` applies $\log(1+x)$ to the four count columns, so the truth is linear in $\log(1+x)$ and not in x. That is not a convenience: it is how the evidence behaves. Going from 0 to 4 reallocated sectors is a **17 times** larger move than going from 40 to 44 ($\log 5 = 1.61$ against $\log(45/41) = 0.09$), because the first sector a drive ever reallocates says something and the forty-first says almost nothing new. In the units the model works in, those first four sectors are worth +4.7 **log-odds**: a drive with average telemetry and no reallocated sectors has a 0.012% chance of failing, and four sectors take it to 1.3% — **about a hundredfold increase**. The four sectors from 40 to 44 are worth +0.27, and move the same drive from 85% to 89%.

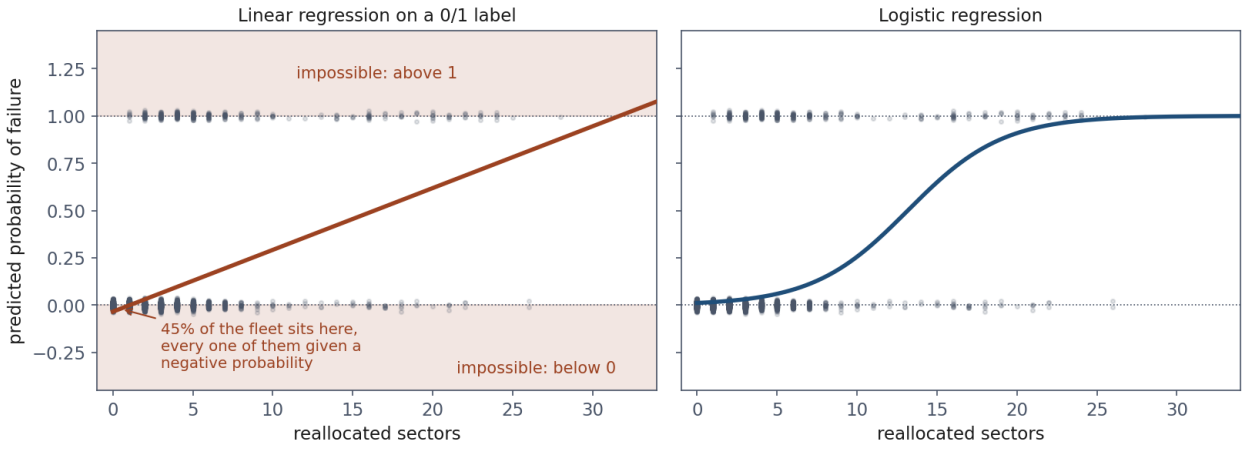
This is the feature engineering of lesson 2, and it has two consequences here. The coefficients in section 2.4 are per standard deviation of the *transformed* column. And the single-feature curve in section 2.1 is fitted on the raw count instead, which is exactly why it is steepest in the middle of the observed range rather than at its left edge.

---

2. The model: from a line to a probability

2.1 What goes wrong with a straight line

Fit ordinary least squares to the 0/1 label using one feature, the count of reallocated sectors. Nothing prevents it. The result is



Left, ordinary least squares on a 0/1 label: the shaded bands are where the line claims something that cannot be true, and 45% of the fleet sits in the lower one. Right, the same data with a curve that cannot leave the interval.

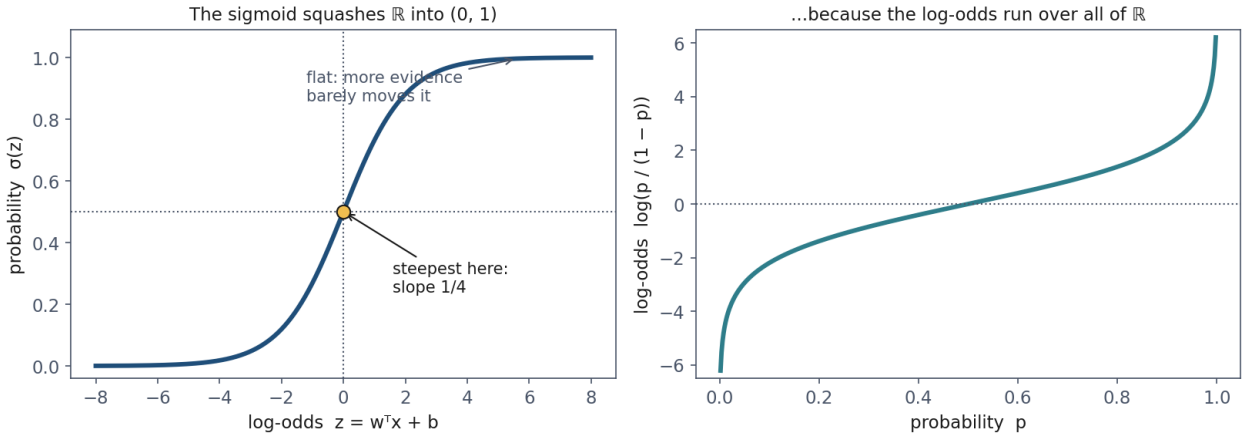
$$\hat{y} = 0.0326\,x - 0.0342$$

and it assigns **a negative probability to 3,606 of the 8,000 drives** — 45% of the fleet. Extend it slightly past the observed range, to 32 reallocated sectors, and it predicts a probability above 1.

This is not a defect that better fitting would remove. A straight line is unbounded and a probability lives in $[0, 1]$; no choice of w and b reconciles the two. **The shape of the function has to change, not its parameters.**

2.2 The sigmoid

The picture first. We want a function that takes any real number — the output of a linear model, which can be anything — and returns something between 0 and 1. It should be **monotone**, so more evidence never lowers the probability. And it should **flatten at both ends**, because evidence has diminishing returns: the same increment of evidence should count for a great deal while the question is still open, and for almost nothing once it is settled.



Left, the sigmoid: steep where the evidence is genuinely ambiguous, flat where your mind is already made up. Right, the same relationship inverted — the log-odds run over the whole real line, which is exactly the range a linear model can safely predict.

The function with those properties is the **sigmoid** or **logistic function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The model becomes

$$\hat{y} = P(y = 1 \mid x) = \sigma(w^\top x + b)$$

Three properties, each of which we will use:

$$\sigma(0) = \tfrac{1}{2}, \qquad \sigma(-z) = 1 - \sigma(z), \qquad \sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

The derivative is the one that matters most. It is largest at $z = 0$, where $\sigma' = \frac{1}{4}$, and it decays to zero at both ends. Section 4 shows that this is simultaneously the reason the sigmoid works and the reason squared error does not.

The diminishing returns, measured. They are diminishing in the *total* evidence z, not in any single counter. Two units of log-odds spent from $z = 0$ move the probability from 0.50 to 0.88; the same two units spent from $z = 6$ move it from 0.9975 to 0.9997 — **178 times less**. The curve is steep where the model is genuinely undecided and flat where its mind is made up.

A predictable mistake, and the figure invites it. Read that against the right-hand panel of the figure in section 2.1, whose horizontal axis is a raw counter rather than z, and the two appear to contradict each other: that curve is steepest at **13.2 reallocated sectors**, and the first four sectors move it by 0.03 against the 0.31 that sectors 10 to 14 are worth. Both pictures are correct. A flat left tail does not say the first four sectors carry no information — it says that on the scale that panel uses, four sectors have not yet made the question interesting. Section 1.1 gives the scale the counters are really measured on, where those same four sectors are worth +4.7 in log-odds.

Deriving the third property, since it is used repeatedly. Write $\sigma = (1 + e^{-z})^{-1}$ and differentiate:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)\,(1 - \sigma(z))$$

using $\dfrac{e^{-z}}{1 + e^{-z}} = 1 - \dfrac{1}{1 + e^{-z}}$ in the last step.

2.3 Odds and log-odds: why this particular curve

The sigmoid is not an arbitrary S-shape. Invert it.

The picture first. A probability is awkward to model linearly because it is trapped between 0 and 1. The **odds** — the probability that something happens divided by the probability that it does not — remove the ceiling: a probability of 0.2 is odds of 0.25, and as the probability approaches 1 the odds run off to infinity. But odds still have a floor at 0. Take the logarithm and the floor goes too, leaving a quantity that ranges over the whole real line.

That quantity is exactly what a linear model can safely predict. Setting $p = \sigma(z)$ and solving for z:

$$z = \log \frac{p}{1 - p}$$

so

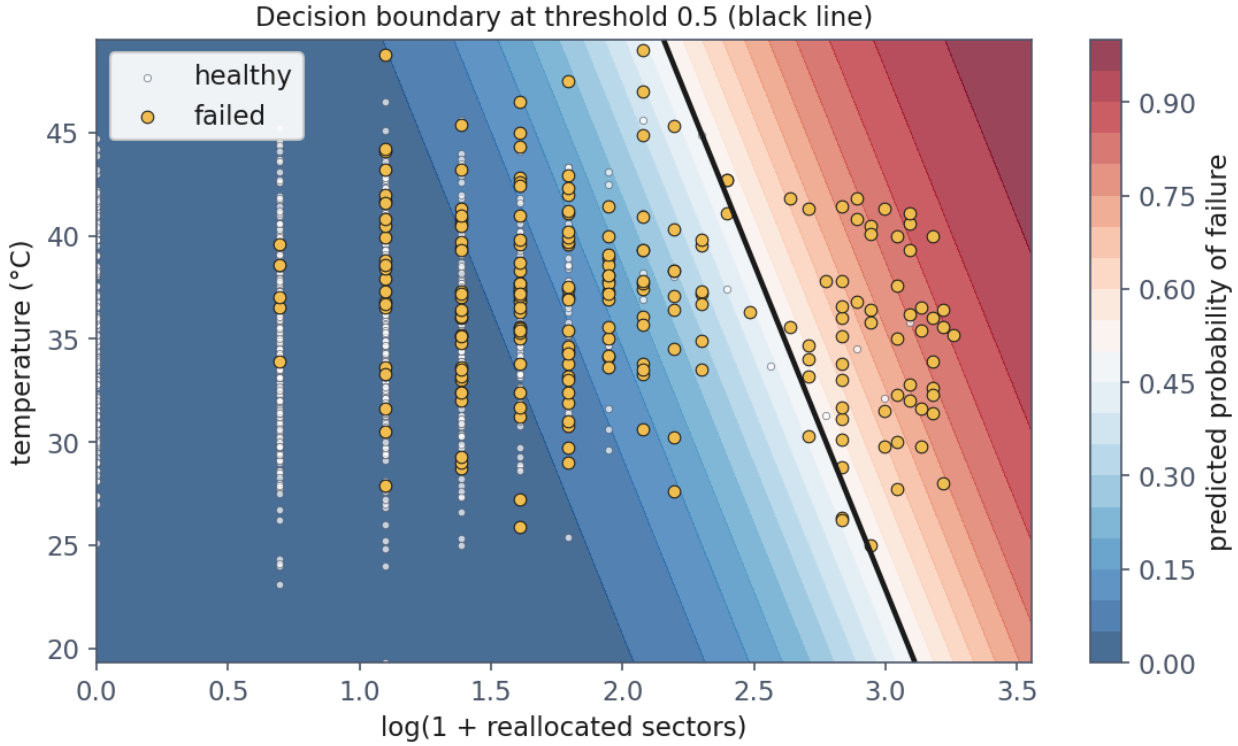
$$\log \frac{P(y = 1 \mid x)}{P(y = 0 \mid x)} = w^\top x + b$$

Logistic regression is a linear model of the log-odds. The sigmoid is not a squashing trick bolted onto linear regression; it is what you get when you model the log-odds linearly and then ask for the probability back.

Probability	Odds	Log-odds
0.01	1 to 99	−4.60
0.10	1 to 9	−2.20
0.50	1 to 1	0
0.90	9 to 1	+2.20
0.99	99 to 1	+4.60

2.4 Reading the coefficients

Because z is log-odds, e^{w_j} is a **multiplier on the odds** for a one-unit increase in x_j. With standardised features, one unit is one standard deviation — of the transformed column, for the four counters that section 1.1 puts on a logarithmic scale.



Two features, so the model can be drawn. The boundary is a straight line — logistic regression is a linear classifier, and the curve is in the probabilities rather than in the boundary. Note how many gold points sit on the healthy side: that is the threshold, not the model.

Our fitted model gives, against the truth used to generate the data:

Feature	True w_j	Fitted	Odds multiplier
reallocated_sectors	1.80	1.66	5.28
spin_retry_count	1.15	1.17	3.22
read_error_rate	0.85	0.87	2.39
temperature_c	0.45	0.47	1.59
power_on_hours	0.35	0.38	1.46

Feature	True w_j	Fitted	Odds multiplier
seek_error_rate	**0.00**	**0.02**	**1.03**
intercept	−6.20	−6.09	—

Two things to read off it.

The decoy was caught. `seek_error_rate` was generated with no effect, and the model estimates it at 0.02, an odds multiplier of 1.03. It did not fall for a plausible-looking column. The harder question — would you have noticed without being told? — is what lesson 5 is about.

The intercept is the average drive's chance, not the base rate. $\sigma(-6.09) = 0.0023$: a drive with perfectly average telemetry has a 0.2% chance of failing in thirty days, while the fleet as a whole fails at 3.8%. The gap is the point: failures come from the small group of drives far above average on one or more counters, not from a typical drive. To reach an even bet, the evidence must move the log-odds by more than six, and only badly degraded drives do that. **That single number explains why every metric in section 5 behaves the way it does.**

A predictable mistake, and the reasonable thinking behind it. Most students read "odds multiplier 5.28" as "five times as likely to fail", and the instinct is sound — for rare events the odds and the probability are nearly the same number, so the reading is *almost* right here. It stops being right as probabilities grow: odds of 9 (probability 0.90) multiplied by 5 gives odds of 45, a probability of 0.978. The odds went up fivefold; the probability moved by under eight points. **Odds ratios multiply odds, not probabilities.**

---

3. Cross-entropy, derived from maximum likelihood

3.1 The idea before the algebra

The picture first. We have a model with a knob — the parameters — and each setting of the knob assigns a probability to every drive failing or surviving. Some settings assign high probability to what actually happened; others assign low probability to it. **Maximum likelihood says: turn the knob to whichever setting makes the data we actually observed as unsurprising as possible.**

That is the whole idea. The algebra below turns "as unsurprising as possible" into a cost function, and the pleasant surprise is that the cost function it produces is not something anyone had to invent — it falls out.

3.2 The likelihood

Write $p^{(i)} = \sigma(w^\top x^{(i)} + b)$ for the model's probability that drive i fails. Each drive contributes one factor: $p^{(i)}$ if it failed, $1 - p^{(i)}$ if it did not. Both cases fit in one expression, because $y^{(i)}$ is 0 or 1 and anything to the power 0 is 1:

$$P(y^{(i)} \mid x^{(i)}) = \left(p^{(i)}\right)^{y^{(i)}} \left(1 - p^{(i)}\right)^{1-y^{(i)}}$$

Assuming the drives are independent given their telemetry, the probability of the whole dataset is the product:

$$\mathcal{L}(w,b) = \prod_{i=1}^{m} \left(p^{(i)}\right)^{y^{(i)}} \left(1-p^{(i)}\right)^{1-y^{(i)}}$$

3.3 Why we take the logarithm

Two reasons, one mathematical and one entirely practical.

Mathematically, the logarithm turns the product into a sum, and sums differentiate one term at a time. It does not move the maximum, because log is strictly increasing.

Practically, the product underflows. Gradient descent starts from all-zero weights, where every one of the 6,000 training drives gets probability 0.5, and 0.5^{6000} is about 10^{-1806}. The smallest positive number a 64-bit float can hold is about 10^{-308}, so the product is stored as exactly zero, and so is every gradient computed from it. Even the fitted model, at an average log loss of 0.0692, multiplies out to only about 10^{-180}. Every serious implementation works in log space for this reason, not for elegance.

Taking the logarithm and negating — so that we minimise rather than maximise, matching the convention of lesson 3 — gives the **binary cross-entropy** or **log loss**:

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m}\left[\, y^{(i)} \log p^{(i)} + \left(1-y^{(i)}\right) \log\left(1-p^{(i)}\right)\right]$$

The single-example loss is

$$L(p,y) = \begin{cases} -\log p & \text{if } y = 1 \\ -\log(1-p) & \text{if } y = 0 \end{cases}$$

Read it as a sentence: **the cost of an outcome is the surprise of seeing it.** An event you assigned probability 1 costs nothing; one you assigned probability 0.001 costs $\log 1000 \approx 6.9$; one you declared impossible costs infinity. That is why log loss is unbounded, and section 4 is about why that is exactly what we want.

3.4 The gradient, and a small miracle

Differentiate the loss for a single example with respect to $z = w^{\top}x + b$. Take $y = 1$, so $L = -\log \sigma(z)$:

$$\frac{\partial L}{\partial z} = -\frac{1}{\sigma(z)} \cdot \sigma'(z) = -\frac{\sigma(z)\,(1-\sigma(z))}{\sigma(z)} = \sigma(z) - 1 = p - y$$

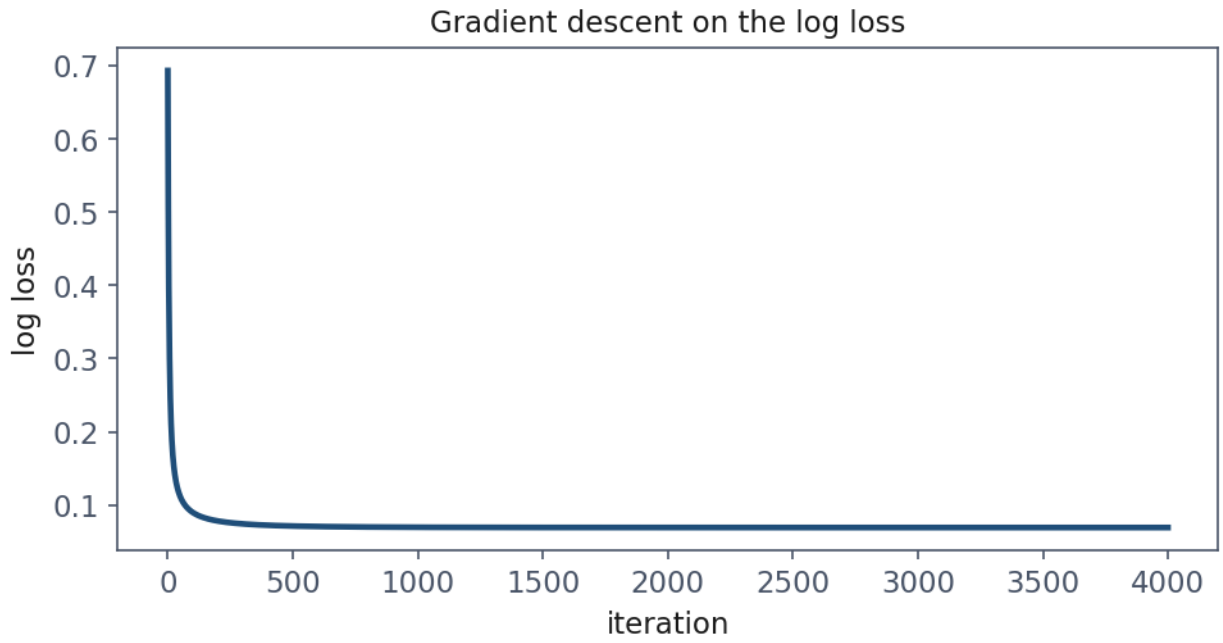
The $\sigma(1-\sigma)$ from the chain rule cancels against the $1/\sigma$ from the logarithm. For $y = 0$ the same cancellation gives $\partial L/\partial z = p$, which is again $p - y$. So in both cases

$$\frac{\partial L}{\partial z} = p - y \qquad \Longrightarrow \qquad \frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right) x_j^{(i)}$$

This is character-for-character the gradient of lesson 3. Prediction error times feature, averaged. A different model and a different cost function produce an identical update rule.

That is not a coincidence, and it is worth knowing why: both are **generalised linear models** fitted by maximum likelihood, linear regression assuming Gaussian noise and logistic regression assuming Bernoulli outcomes. For the whole exponential family the maximum-likelihood gradient takes this form. You do not need that theory to use either method — but it explains why the same gradient descent code, unchanged, trains both.

The same code, run on the drives. Notebook 1 implements that update in fifteen lines and runs it for 4,000 iterations on the six standardised features, starting from $w = 0$ and $b = 0$. The log loss falls from 0.6931 to 0.0692.



What to look at: the descent is over almost at once. The first 100 iterations cover 96.7% of the total fall, and the last 100 of the 4,000 change the loss by nothing at the fifth decimal — the run is long out of caution, not necessity.

The starting value is a free test. With $w = 0$ and $b = 0$ the model gives every drive $p = 0.5$, so the loss must begin at $-\log \frac{1}{2} = \log 2 = 0.6931$, which is what the notebook prints. **A from-scratch implementation that starts anywhere else has a bug in its initialisation** — one number, checked before any of the training matters.

---

4. Why not squared error

4.1 The cost of being confidently wrong

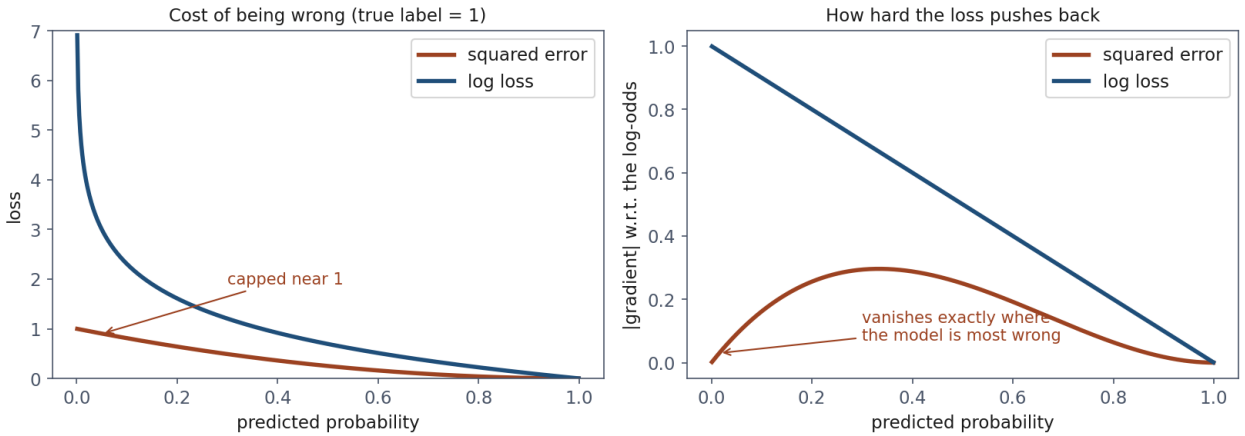
Suppose a drive failed and the model said 0.0001.

Loss	Value
Squared error $(p-1)^2$	0.9998
Log loss $-\log p$	9.21

Squared error charges **0.9998 for a confident falsehood and 0.25 for an honest "don't know"**. A factor of four separates the worst possible answer from a shrug. Log loss charges 9.21 against 0.69, a factor of thirteen, and the ratio grows without bound as $p \to 0$.

4.2 The sharper problem: the gradient dies

The costs matter less than the gradients, because gradients are what the optimiser uses. Differentiate both with respect to z, for an example with $y = 1$:



Left, what each loss charges for being wrong: squared error is capped near 1, log loss is not. Right, the part that matters — squared error's gradient collapses towards zero exactly where the model is most confidently wrong, while log loss pushes hardest there.

$$\frac{\partial}{\partial z}(p-1)^2 = 2(p-1)\,p\,(1-p) \qquad\qquad \frac{\partial}{\partial z}\left[-\log p\right] = p - 1$$

The squared-error gradient carries an extra factor $p(1-p)$, which is $\sigma'(z)$ — and σ' vanishes at both ends.

p	|grad|, squared error	|grad|, log loss
0.5	0.250	0.500
0.1	0.162	0.900
0.01	0.0196	0.990
0.001	0.0020	0.999

As the model becomes more confidently wrong, squared error asks it to change less. At $p = 0.001$ the gradient is a five-hundredth of the log loss gradient. Log loss goes the other way: its gradient approaches its maximum exactly where the error is worst.

The cancellation in section 3.4 is the same fact seen from the other side. Log loss is *designed*, by maximum likelihood, so that the sigmoid's vanishing derivative cancels out. Squared error is not, so it does not.

4.3 Convexity

There is a third reason, and section 3.4 has already done most of the work for it.

J as defined in section 3.3 is **convex** in (w, b), which takes two steps to see. Section 3.4 gave the gradient, $\partial J/\partial w = \frac{1}{m}\sum_i (p^{(i)} - y^{(i)})\, x^{(i)}$. Differentiating it once more, the only thing that depends on w is $p^{(i)}$, and by the chain rule through $z^{(i)} = w^\top x^{(i)} + b$ its derivative is $p^{(i)}(1-p^{(i)})\, x^{(i)}$. So the matrix of second derivatives is

$$H = \frac{1}{m}\sum_{i=1}^{m} p^{(i)}\left(1-p^{(i)}\right) x^{(i)}\left(x^{(i)}\right)^\top = \frac{1}{m} X^\top S X, \qquad S = \operatorname{diag}\left(p^{(i)}(1-p^{(i)})\right)$$

Convexity is then one line. A matrix is positive semi-definite when $u^\top H u \geq 0$ for every direction u, and here

$$u^\top H u = \frac{1}{m}\sum_{i=1}^{m} p^{(i)}\left(1-p^{(i)}\right)\left(\left(x^{(i)}\right)^\top u\right)^2 \geq 0$$

because every term is a positive number, $p(1-p)$, multiplying a square. There is no direction in which the surface curves downwards. Any local minimum is therefore global, and gradient descent cannot be trapped.

Squared error composed with the sigmoid has no such guarantee, and is in general non-convex in w. So the choice of cost function is not a matter of taste: it determines whether the optimisation problem is one we know how to solve.

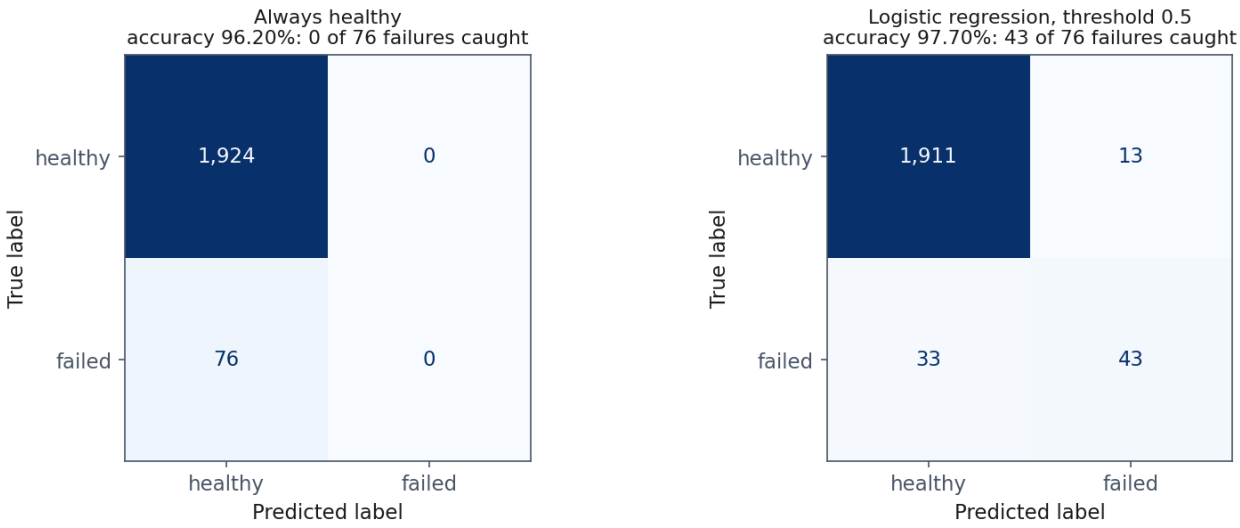
A caution students reasonably get wrong. Convex does not mean there is a closed-form solution. Setting the gradient to zero gives $X^\top(\sigma(Xw+b) - y) = 0$, which is not linear in w — the sigmoid is in the way — and has no algebraic solution. There is **no normal equation for logistic regression**. It must be solved iteratively, which is why `LogisticRegression` has a `max_iter` argument and `LinearRegression` does not.

---

5. The trap: accuracy

5.1 The number to remember from this lesson

Our test set is 2,000 drives, 76 of which failed. Consider two models.



The same test set under two models. The accuracy figures differ by 1.5 points; the number that separates them — 0 failures caught against 43 — appears in neither.

Model	Accuracy	Failures caught
Predict "healthy" for everything	**96.20%**	**0 of 76**
Logistic regression, threshold 0.5	97.70%	43 of 76

96.20% accuracy, nothing caught. A model containing no learning of any kind, which could be written as `return 0`, is within 1.5 percentage points of a model that finds more than half the failing drives.

The arithmetic is not subtle. 1,924 of the 2,000 drives are healthy, so answering "healthy" collects 1,924 correct answers for free. Only 3.8% of the test set is left to compete over, and no amount of skill moves accuracy far.

5.2 Why this keeps happening

This is the predictable mistake of the lesson, and the reasoning behind it is completely sound: accuracy is the fraction of predictions that were correct, and on a balanced problem that is exactly what you want to know. It is not a naive metric.

It stops being informative precisely when one class is rare — which is to say, on most problems anyone builds a classifier for. Fraud, disease screening, equipment failure, intrusion detection, spam: **the interesting class is always the small one.** Accuracy is not wrong. It is answering a question nobody asked.

The general statement: with a positive rate π, the trivial always-negative classifier scores $1 - \pi$. At $\pi = 0.038$ that is 96.2%; at $\pi = 0.001$, as in some fraud problems, it is 99.9%.

---

6. The confusion matrix and what is built on it

Section 5 showed that one number cannot describe a classifier on imbalanced data. The remedy is not a better single number — it is to stop summarising prematurely. Every metric in this section is built from the same four counts, and the counts come first because which metric you should care about is a question about *which of the four mistakes hurts*, and no formula answers that for you.

6.1 Four numbers

Stop collapsing the result into one number and count all four outcomes.

	Predicted healthy	Predicted failing
Actually healthy	TN — true negative	FP — false positive
Actually failed	FN — false negative	TP — true positive

The names mean nothing until attached to consequences, so attach them:

- **False positive**: we replace a healthy drive. Cost: a technician's hour and the price of a drive.
- **False negative**: a drive fails in service unwarned. Cost: an emergency callout, a degraded array, possibly data.

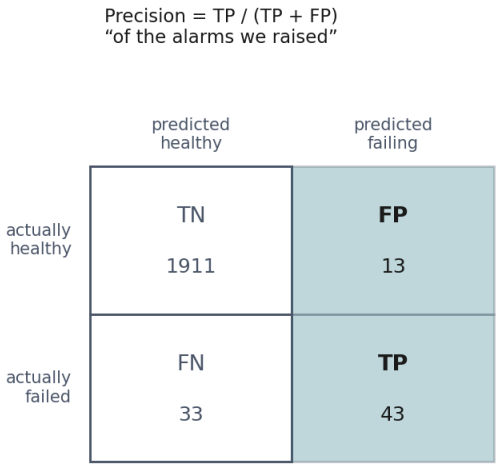
For our model at threshold 0.5:

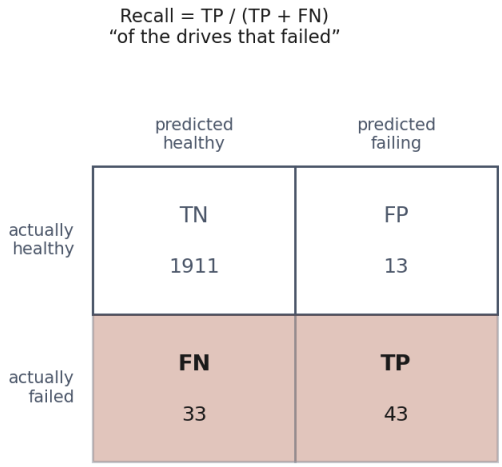
	pred. healthy	pred. failing
healthy	1911	13
failed	33	43

Read the bottom-left cell first when misses are expensive: **33 drives failed and were called healthy.**

6.2 Precision and recall

Both divide the true positives. They differ in the denominator, and the denominator is the whole meaning.





*The difference is entirely the denominator. Precision divides by the shaded **column**, everything we flagged; recall divides by the shaded **row**, everything that actually failed.*

$$\text{precision} = \frac{TP}{TP + FP} \qquad \text{recall} = \frac{TP}{TP + FN}$$

Precision divides by everything we flagged. *When this model raises an alarm, how often is it right?* It is the technician's question: how much of my time are you wasting?

Recall divides by everything that was truly positive. *Of the drives that were going to fail, how many did we find?* It is the operations manager's question: how exposed am I still?

A hook that survives exam pressure: **precision is about the alarms, recall is about the failures.** Look at the denominator and ask which population you are dividing by.

For our model: precision 0.768, recall 0.566. Of the 56 drives flagged, 43 really failed; of the 76 that failed, we found 43.

Two further quantities appear often enough to name:

$$\text{specificity} = \frac{TN}{TN + FP} = 0.993 \qquad \text{false positive rate} = 1 - \text{specificity} = 0.007$$

6.3 F1, and why the harmonic mean

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

That expression is the **harmonic mean** of the two, which is easier to recognise in the form it is defined by — invert both, average, invert back:

$$\frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

multiplying numerator and denominator by the product of the two. Inverting is what makes it behave differently from an ordinary average: a small number has a large reciprocal, so the *worse* of the two dominates the sum in the denominator.

Why not the ordinary average? Consider the model that flags every drive. Recall is a perfect 1.000; precision is 0.038.

Mean	Value
Arithmetic	0.519
Harmonic (F_1)	0.073

The arithmetic mean gives a useless model a respectable-looking score, because a perfect result on one axis pays for a catastrophe on the other. The harmonic mean is dominated by the smaller number: **a model is only as good as its weaker side.**

The generalisation, when the two are not equally important, is the same reciprocal average with the two reciprocals given unequal weights — 1 and β^2 out of $1 + \beta^2$:

$$F_\beta = \frac{1+\beta^2}{\frac{1}{\text{precision}} + \frac{\beta^2}{\text{recall}}} = (1+\beta^2)\,\frac{\text{precision}\cdot\text{recall}}{\beta^2\cdot\text{precision}+\text{recall}}$$

Setting $\beta = 1$ gives equal weights and recovers F_1, which is the check worth doing on any weighted formula. $\beta > 1$ puts more weight on recall's reciprocal, so a poor recall hurts more — F_2 is the usual choice when misses are costly, as here. $\beta < 1$ favours precision. The square is there so that β reads as a ratio of importance rather than of reciprocals: at $\beta = 2$, recall is worth four times precision in the sum, which is the convention the name carries.

6.4 Averaging across classes

Scikit-learn's `classification_report` — the summary table notebook 2 prints, one row per class followed by three summary rows — is where the numbers below come from. Choosing between those three rows matters more than it looks.

Average	What it does	Our model
macro	mean over classes, each equal	0.820
weighted	mean over classes, weighted by support (the number of examples in each class)	0.975
micro / accuracy	pool all predictions	0.977

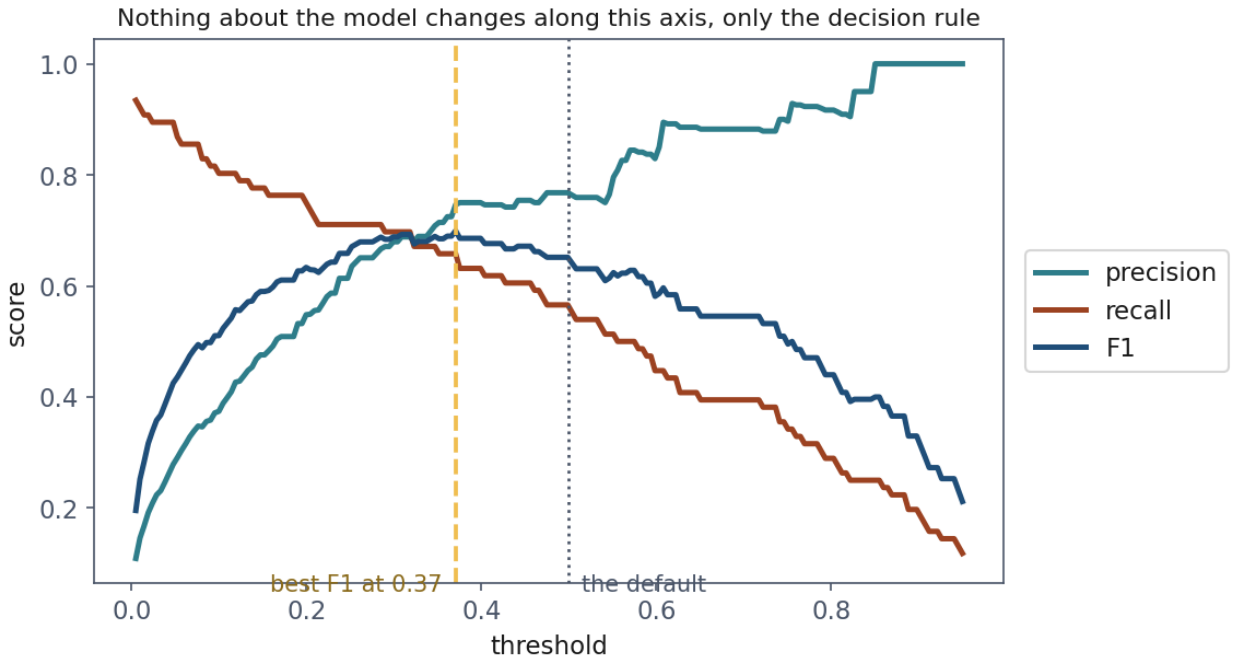
The weighted average is dominated by the healthy class, which is five drives in six, so it reads close to accuracy and inherits its blindness. **On an imbalanced problem, quote the macro average or the minority class directly.** Reporting the weighted average is not incorrect, but it is the number that makes a mediocre model look finished.

---

7. The threshold is a decision

7.1 Nothing about the model changes along this axis

Every number in section 6 depended on a rule we never examined: flag the drive when $\hat{y} \geq 0.5$.



The fitted model is identical at every point on this axis: same coefficients, same probabilities, same drives. Only the line between "leave it" and "replace it" moves.

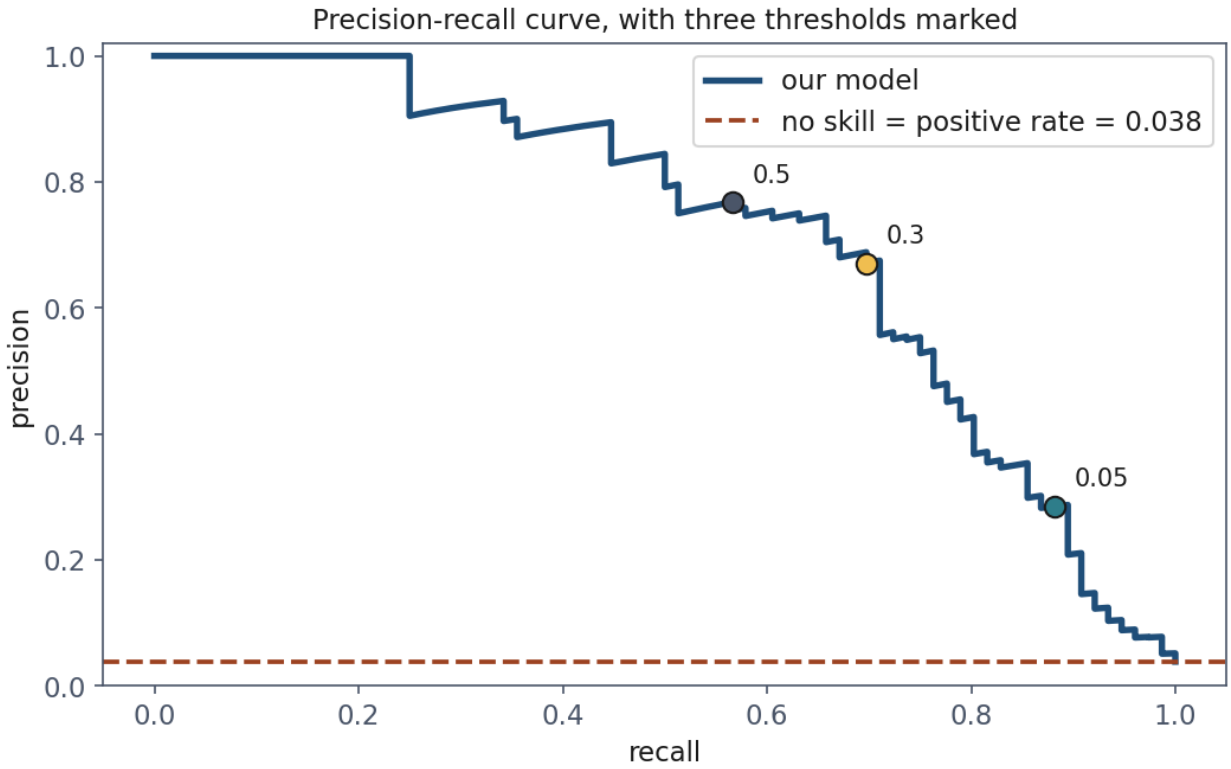
That 0.5 is `predict()`'s default and nothing more. It is the right choice when the two errors cost the same *and* the classes are balanced — neither of which holds here. The model's actual output is a probability; **turning it into a decision requires a threshold, and the threshold is a business question.**

Threshold	Flagged	TP	FP	FN	Precision	Recall	F_1	Accuracy
0.02	353	69	284	7	0.195	0.908	0.322	0.855
0.05	235	67	168	9	0.285	0.882	0.431	0.911
0.10	163	61	102	15	0.374	0.803	0.510	0.942
0.20	104	57	47	19	0.548	0.750	0.633	0.967
0.30	78	53	25	23	0.679	0.697	**0.688**	0.976
0.50	56	43	13	33	0.768	0.566	0.652	**0.977**
0.70	34	30	4	46	0.882	0.395	0.545	0.975
0.90	14	14	0	62	1.000	0.184	0.311	0.969

The same fitted model produced every row. Same coefficients, same probabilities, same drives. Only the line between "leave it" and "replace it" moved.

Note the accuracy column once more. From 0.20 upwards it varies by one percentage point while recall falls from 0.750 to 0.184, and its maximum sits at the default threshold, which misses 33 failures. **Tuning on accuracy quietly recommends catching fewer failures**, because every alarm it drops was a potential false positive.

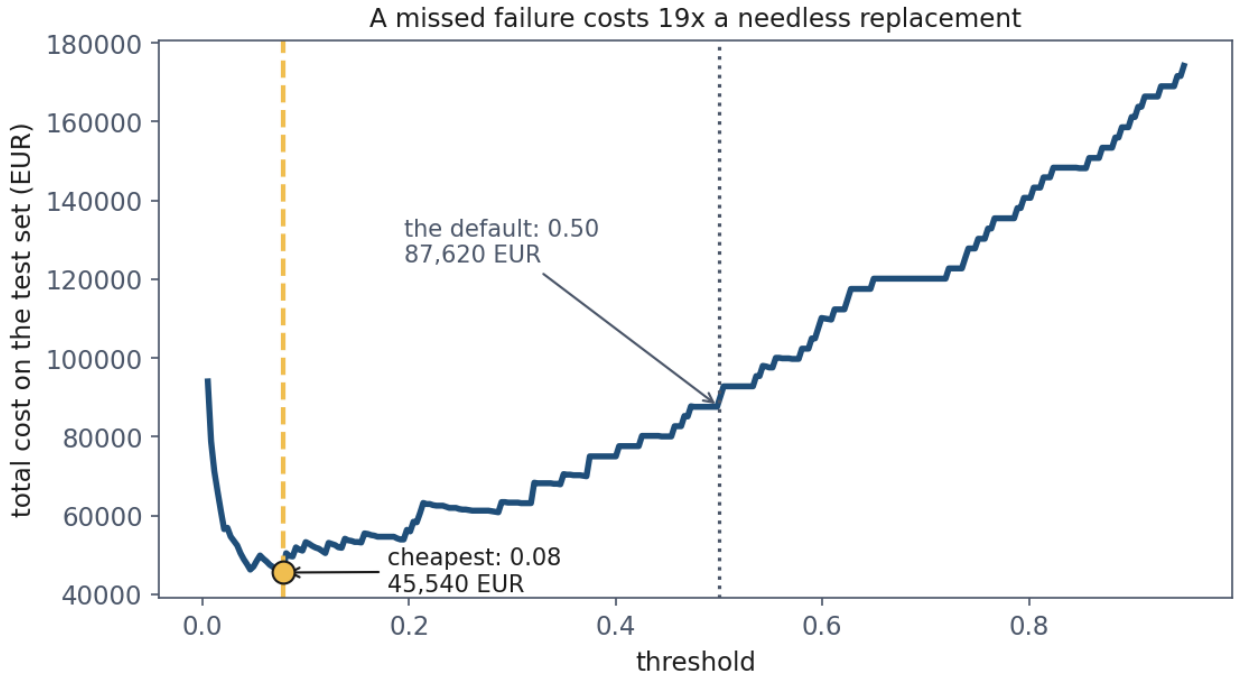
So when a paper or a colleague reports 77% precision, the useful question is: *at what threshold, and why that one?*



The table above, drawn as a curve, with three of its rows marked. Each point is one threshold; moving along the curve is the trade the table tabulates. The dashed line is what "no skill" looks like on this problem — the positive rate, 0.038, not 0.5 — and Section 8 returns to why that baseline matters more than the curve's shape.

7.2 Choosing the threshold from costs

The only defensible answer requires one ingredient no metric supplies: **what the errors cost.** Suppose



Total cost against threshold, with a miss priced at nineteen false alarms. The minimum is far below 0.5, and the curve is nearly flat around it — so the exact value does not have to be right, which is fortunate, because cost estimates never are.

$$C_{FP} = 140 \text{ EUR} \qquad C_{FN} = 2600 \text{ EUR}$$

A missed failure costs about nineteen needless replacements. Now the threshold follows from arithmetic rather than habit.

The derivation. For a drive with estimated failure probability p, flagging it has expected cost $(1-p)\,C_{FP}$ — we pay for a replacement only if the drive was in fact healthy. Leaving it alone has expected cost $p\,C_{FN}$. Flag when flagging is cheaper:

$$(1-p)\,C_{FP} < p\,C_{FN}$$

Multiply out the left-hand side and collect the two terms in p on one side: $C_{FP} < p\,C_{FN} + p\,C_{FP} = p\,(C_{FN} + C_{FP})$, so

$$p > \frac{C_{FP}}{C_{FP} + C_{FN}} = t^*$$

$$t^* = \frac{140}{140 + 2600} = 0.051$$

Read the formula before the number. The optimal threshold does not depend on the model, the dataset, or the class balance — only on the *ratio* of the two costs. If misses and false alarms cost the same, $t^* = 0.5$ and the default is correct. As misses grow more expensive, t^* falls towards zero.

On the test set, sweeping the threshold empirically puts the cheapest value at **0.08**, close to the theoretical 0.051, and the cost curve is nearly flat between them:

Policy	Cost on 2,000 drives	FP	FN
Do nothing	197,600 EUR	0	76
Threshold 0.50	87,620 EUR	13	33
Threshold 0.08	**45,540 EUR**	121	11

Choosing the threshold on purpose halved the cost of the same model. No retraining, no new features, no better algorithm — one number, chosen from the economics instead of from the library default.

Two honest caveats.

The flatness is a feature. Because the curve is nearly flat near its minimum, the exact threshold does not have to be right. That is fortunate, because cost estimates never are.

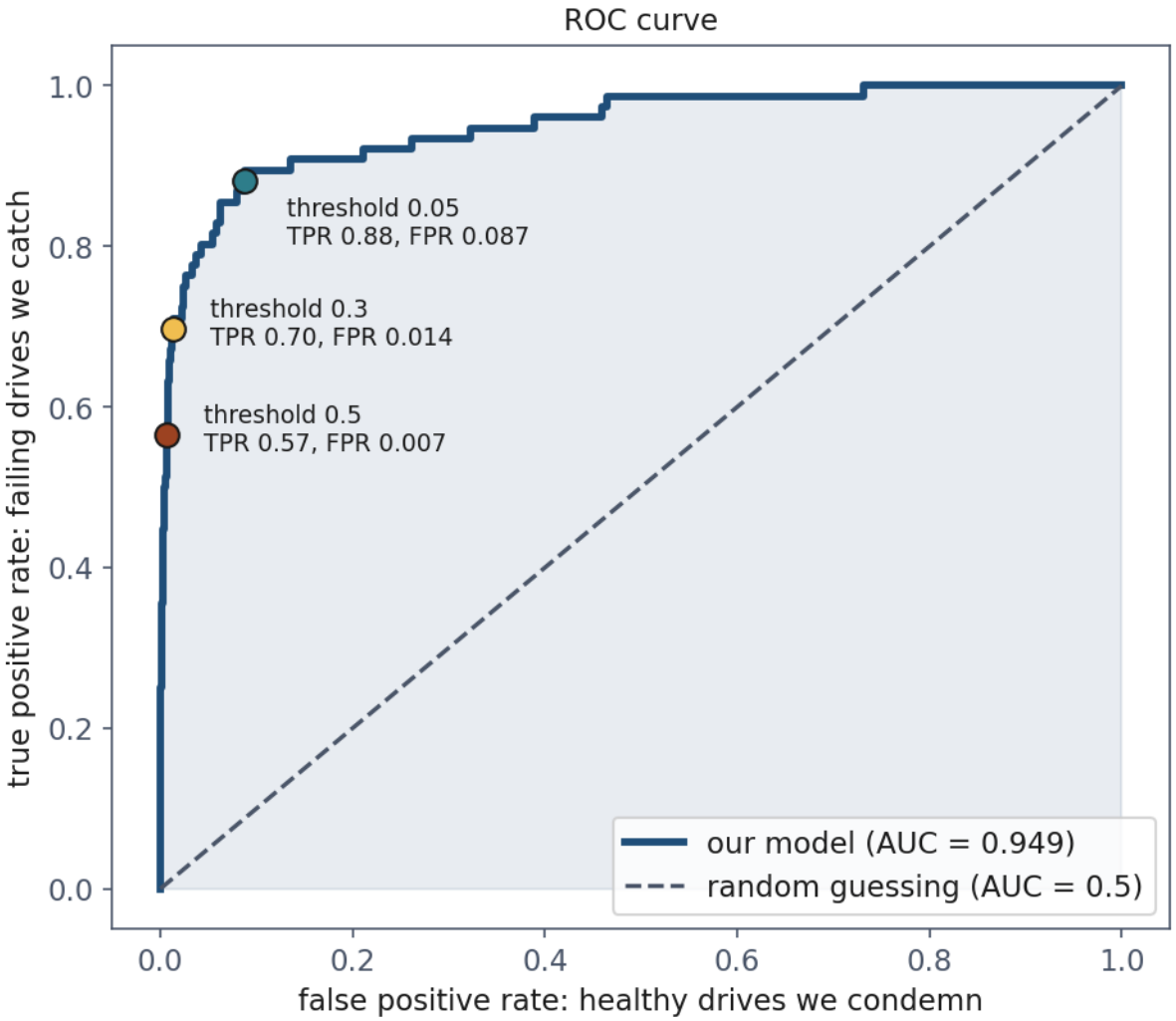
This threshold was chosen on the test set, which is exactly what lesson 1 told you not to do. Properly, it is chosen on a validation set and only then measured on test data. Lesson 5 builds that machinery; until then this is a demonstration of a method, not a number you could report.

---

8. ROC and AUC

8.1 The curve

Rather than pick a threshold, sweep all of them and plot two rates against each other:



The three marked points are three thresholds on one curve. The diagonal is guessing; the top-left corner is perfection; the area between says how far towards the corner this model got.

$$\text{TPR} = \frac{TP}{TP + FN} = \text{recall} \qquad \text{FPR} = \frac{FP}{FP + TN}$$

The **true positive rate (TPR)** is the fraction of failing drives caught; the **false positive rate (FPR)** is the fraction of healthy drives wrongly condemned. The plot is the **receiver operating characteristic (ROC)** curve — the name is wartime radar, where operators tuned receivers to spot aircraft without chasing flocks of birds.

The intuition. At threshold 1 both rates are 0: flag nothing, catch nothing, disturb nobody. At threshold 0 both are 1. The whole content of the curve is *the order in which they get there.* A good model raises TPR fast while FPR is still low, because its highest scores really are the failures. A useless model raises them together and traces the diagonal.

So the curve asks: **as we grow more willing to raise alarms, do we catch failures faster than we annoy technicians?**

8.2 What the area means

The **area under the curve (AUC)** summarises the whole curve in one number: 0.5 for guessing, 1.0 for perfection. Ours is **0.949**.

The area has an exact probabilistic meaning, and it is the thing worth remembering:

> **AUC is the probability that the model assigns a higher score to a randomly chosen positive than to a randomly chosen negative.**

That is, pick one drive that failed and one that did not, at random; AUC is the probability the failing one scored higher.

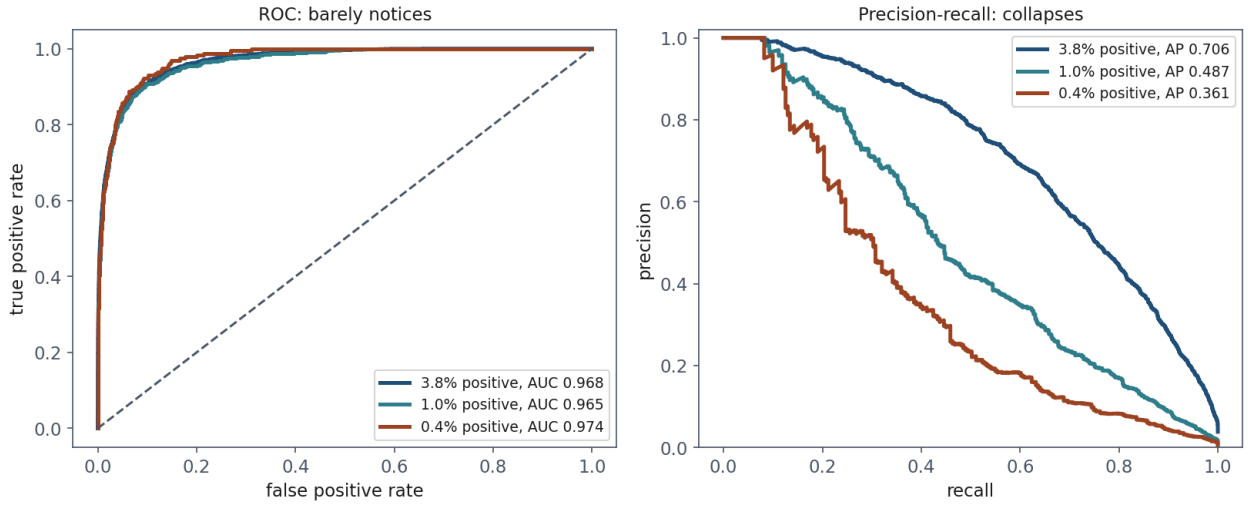
A sketch of why. Order all $m_+ m_-$ pairs of one positive and one negative. Sweeping the threshold downwards, each step to the right on the ROC plot admits one negative and each step upwards admits one positive; the area accumulated under the curve counts exactly the pairs in which the positive was admitted first — that is, scored higher. Dividing by $m_+ m_-$, the total number of pairs, gives the probability. This quantity is the Mann-Whitney U statistic, and the identity is standard.

Notebook 3 checks it by experiment rather than accepting the argument: drawing 200,000 random (failing, healthy) pairs, the failing drive scored higher in **0.9488** of them, against `roc_auc_score`'s **0.9493**.

A consequence that catches people out. AUC depends only on the *ranking*, so it is invariant to any monotone transformation of the scores. Divide every predicted probability by 10 and the AUC is unchanged, while the model becomes badly calibrated and every threshold-based decision changes. **AUC says nothing about whether "0.7" means anything.** If you need the probabilities to be believable — because you are computing expected costs, as in section 7.2 — AUC is not the check you want.

8.3 Where ROC misleads

An AUC of 0.949 sounds like a finished project. Now make the positives rarer, which is the direction reality moves in fraud, disease and hardware failure.



One model, drawn twice, as the positives are thinned from 3.8% to 0.4%. On the left the three curves lie almost on top of each other. On the right they are in different worlds.

Keep the same model, keep every healthy drive in a fleet of 60,000, and thin the failures until they are 0.4% instead of 3.8%. **The model is not retrained.**

Positive rate	AUC	Average precision	Precision at 0.5
3.8%	0.968	0.706	0.788
1.0%	0.965	0.487	0.462
0.4%	0.974	0.361	0.259

AUC does not move. It is a property of the ranking, and thinning positives at random does not change how the model ranks. It goes on reporting an excellent model — at the rarest setting, a marginally better one, which is sampling noise on 231 positives.

Precision falls to a third of what it was. At the same threshold, with the same model, two alarms in three are now false. That is the number the technician experiences.

Why the disagreement. Look at the denominators. FPR divides false positives by the number of healthy drives, which is enormous and growing: a thousand false alarms among a hundred thousand healthy drives is an FPR of 0.01, invisible on the ROC axis. Precision divides those same thousand alarms by the alarms raised, where they dominate.

This is the second predictable mistake of the lesson, and again the instinct is defensible: AUC is threshold-free, comparable across models and datasets, and bounded in a familiar range. Those are real virtues. It simply answers a question about ranking when the question you had was about the alarms someone will act on.

The rule: on an imbalanced problem, report the precision-recall curve alongside AUC, never instead of it. Its baseline is the positive rate — 0.038 here, not 0.5 — and its area, the **average precision**, was 0.717 for our model.

---

9. Class weights and resampling

Moving the threshold changes the decision *after* training. **Class weights** change the training itself, by making each rare example count as several.

The weighted cost is

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m} c_{y^{(i)}}\left[\,y^{(i)}\log p^{(i)} + \left(1-y^{(i)}\right)\log\left(1-p^{(i)}\right)\right]$$

with `class_weight="balanced"` setting $c_k = m/(K\,m_k)$, where m_k is the number of examples of class k — inversely proportional to each class's frequency. Here that makes each failure count about 25 healthy drives.

Model	TP	FP	FN	Precision	Recall	AUC	Cost
Plain, threshold 0.50	43	13	33	0.768	0.566	0.949	87,620

Model	TP	FP	FN	Precision	Recall	AUC	Cost
Plain, threshold 0.08	65	121	11	0.349	0.855	0.949	45,540
Balanced weights, 0.50	68	205	8	0.249	0.895	0.950	49,500

Look at the AUC column: 0.949 against 0.950. Reweighting taught the model essentially nothing new about disk failure. What it did was move where the model puts its 0.5 line — the same lever as the threshold, pulled at a different moment.

That is worth knowing, because class weighting is often presented as a remedy for imbalance. It is not a remedy; it is another way of setting the same decision. The remedy, where one exists, is more positive examples.

Where it earns its place: when a downstream tool insists on `predict()` and gives you no access to a threshold, or when the model's probabilities are not usable. Then weighting is how you move the operating point.

The same caution applies to **resampling** — oversampling the minority class, or synthesising new minority examples with SMOTE (synthetic minority oversampling technique). It changes the effective threshold and can help an optimiser see a rare class at all, but it does not create information that was not in the data. And it must happen **inside the cross-validation fold**, never before the split, for exactly the reasons lesson 2 gave about imputation.

---

10. More than two classes

Real fleets are not healthy-or-dead. A drive can be **degraded**: readable, but reporting errors and worth replacing at the next maintenance window rather than at three in the morning.

10.1 Softmax

For K classes the model produces one score per class and normalises:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad z_k = w_k^\top x + b_k$$

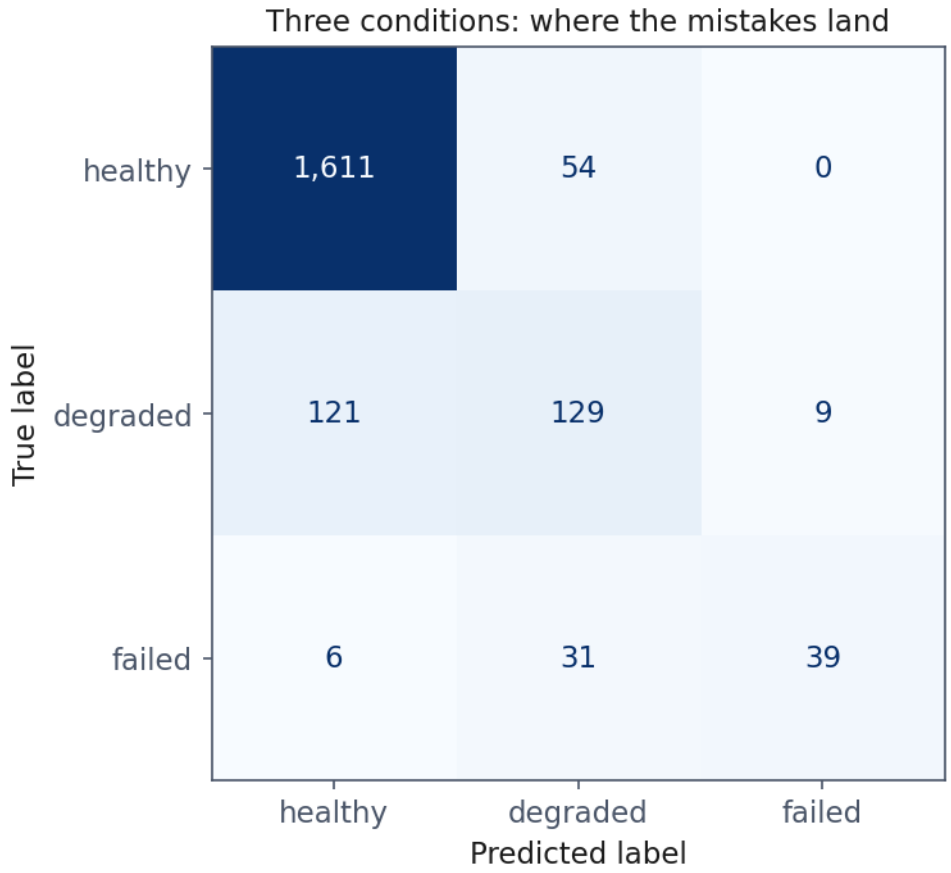
This is the **softmax**, and for $K = 2$ it reduces to the sigmoid — subtract z_0 from both scores and one of them becomes 0, leaving $\sigma(z_1 - z_0)$. The cost generalises to **categorical cross-entropy**:

$$J = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} \mathbb{1}\left[y^{(i)} = k\right] \log P\big(y = k \mid x^{(i)}\big)$$

which is the same idea as before: the negative log of the probability the model assigned to what actually happened.

10.2 Metrics with three classes

The confusion matrix becomes $K \times K$, and precision and recall are computed **one class at a time**: for each class in turn, that class is positive and the rest are negative. On our three conditions:



Read the off-diagonal cells rather than the accuracy. Degraded is the hard class: it sits between two neighbours that both resemble it, and the model confuses it in both directions.

Class	Precision	Recall	F_1	Support
healthy	0.927	0.968	0.947	1665
degraded	0.603	0.498	0.545	259
failed	0.812	0.513	0.629	76
macro avg	0.781	0.660	0.707	
weighted avg	0.881	0.889	0.883	

Read the off-diagonal cells of the matrix, not the accuracy. Degraded is the hard class, and understandably so: it sits between two neighbours that both look like it from a distance.

And the macro–weighted gap is now stark, 0.707 against 0.883. Which to quote is a question about the application: **if missing a failure matters as much as missing a healthy drive, macro is the honest one.**

11. Summary

- A linear model on a 0/1 label predicts values that are not probabilities. The **sigmoid** fixes it by modelling the **log-odds** linearly, and e^{w_j} is a multiplier on the **odds** — not on the probability.
- **Cross-entropy is not a design choice**; it is what maximum likelihood produces. Its gradient, $(\hat{y} - y)x$, is identical to linear regression's.
- **Squared error fails** because its gradient carries a factor $\sigma'(z)$ that vanishes exactly where the model is most confidently wrong, and because it is not convex through the sigmoid.
- **96.20% accuracy, zero failures caught.** On an imbalanced problem accuracy measures the majority class and nothing else.
- The **confusion matrix** is the honest object. **Precision is about the alarms, recall is about the failures.** F_1 uses the harmonic mean so a perfect score on one axis cannot buy a pass on the other.
- The **threshold is a decision**, and the cost-optimal one is $t^* = C_{FP}/(C_{FP} + C_{FN})$ — a function of the costs alone. Choosing it on purpose halved the cost here.
- **AUC is the probability a random positive outranks a random negative.** It measures ranking, is blind to calibration, and **hides imbalance** — report the precision-recall curve beside it.
- **Class weights are the threshold by another name.** Note how little the AUC moved.

Homework

`Exercises/04_classification_and_metrics.md`, discussed at the start of Lesson 5, **Friday 30 October 2026**.

Notation used in this lesson

Symbol	Meaning
x, X	one drive's telemetry; the design matrix
y, $\hat{y}$	true label (0 or 1); predicted probability
$p^{(i)}$	the model's probability that drive i fails: the same number as $\hat{y}^{(i)}$, written p in the derivations
z	the log-odds, $w^\top x + b$
w, b	coefficients, intercept
m, n	number of examples, number of features
J, L	the cost over the dataset; the loss on one example
H	the matrix of second derivatives of J
σ	the sigmoid
π	the positive rate in the data
t^*	the cost-optimal threshold
C_{FP}, C_{FN}	the cost of a false positive, of a false negative